



西安交通大学
XI'AN JIAOTONG UNIVERSITY

机器学习

实验报告

基于 AlphaZero 架构的黑白棋对弈模型

2024-2025 学年 第二学期

学 院： 数学与统计学院
班 级： 数学强基 2202
学 号： 2224110096
姓 名： 陈宇昂
指导教师： 孟德宇老师
实验地点： 数学实验中心
完成时间： 2025-03-30

目录

1	实验内容	1
1.1	问题描述	1
1.2	实验步骤	1
1.3	实验要求	1
2	实验描述	2
2.1	数据集与实验环境参数	2
2.2	算法描述	2
2.2.1	卷积神经网络	3
2.2.2	蒙特卡洛树搜索	5
2.2.3	自对弈训练	6
2.3	模块化设计	7
2.3.1	模型的建立	7
2.3.2	GUI 架构	7
2.3.3	模型的训练和评估	8
2.4	程序框图	9
3	实验结果与所遇问题	9
3.1	实验结果分析	9
3.2	问题与解决措施	10
A	附录：实验代码	12
A.1	Reversi_model.py	12
A.2	play.py	22
A.3	AI_vs_AI.py	26

1 实验内容

1.1 问题描述

本次实验旨在构建一个基于 AlphaZero 架构的 8×8 黑白棋对弈模型。黑白棋是一种策略性棋类游戏，规则相对简单但策略深度较高。AlphaZero 架构结合了神经网络和蒙特卡洛树搜索（MCTS），能够在无监督的情况下通过自对弈提升棋力。我希望训练出一个能够在 8×8 黑白棋棋盘上具有一定智能的模型（能够与人类初学者进行对弈），并讨论了自对弈轮数和每轮迭代次数对于模型的训练效果。

1.2 实验步骤

1. 定义游戏逻辑：在 `Reversi_model.py` 中实现黑白棋的基本规则：棋盘初始化、合法走子判断、落子执行和游戏结束条件判断等。
2. 构建神经网络模型：设计一个卷积神经网络 `ReversiNNet`，用于输出落子概率分布和局面估值。
3. 实现蒙特卡洛树搜索（MCTS）：在 `Reversi_model.py` 中实现 MCTS 算法，辅助模型进行决策。
4. 自对弈数据生成：通过自对弈过程生成训练数据，每个回合记录当前棋盘状态、落子概率和最终胜负结果。
5. 模型训练：使用生成的训练数据对 `ReversiNNet` 进行训练，优化模型参数。
6. 人机对弈和机机对弈：使用 `play.py` 构建 GUI 界面实现人机对弈，使用 `AI_vs_AI.py` 进行机机对弈，测试各模型的棋力。

1.3 实验要求

1. 模型性能：训练出的模型应具有一定的智能，能够与初学者水平的人类棋手一较高下；
2. 代码实现：代码应具有良好的可读性和可维护性，遵循模块化设计原则；
3. 实验记录：记录实验过程中的关键参数和结果，探索模型性能与超参数之间的数量关系。

2 实验描述

2.1 数据集与实验环境参数

数据集： 本实验的数据集是通过模型的自对弈过程生成的。在每次自对弈中，记录当前棋盘状态、落子概率和最终胜负结果，作为训练数据。由于黑白棋的状态空间相对较小，自对弈生成的数据足以训练出一个性能相对较好的模型。

实验环境参数 本次实验的相关参数如表1所示。

表 1: 实验运行相关参数

项目	参数
执行平台	PyCharm
主要依赖库	Pytorch, Numpy, Tkinter
Python 版本	Python 3.11.8
GPU 型号	NVIDIA GeForce RTX 3050 Ti
CUDA 版本	CUDA 12.7

2.2 算法描述

AlphaZero 算法是 DeepMind 研究团队于 2017 年提出的一种基于强化学习的通用博弈算法 [1]，其核心框架由卷积神经网络、蒙特卡洛树搜索（MCTS）和自对弈训练三部分组成。

该算法通过自我博弈生成数据，结合卷积神经网络的策略和价值预测，实现对复杂博弈问题的端到端求解。AlphaZero 算法通过结合卷积神经网络与蒙特卡洛树搜索，实现了博弈问题的高效求解。其核心创新点在于：

1. 统一架构：同一模型适用于多种不同博弈类型（围棋、国际象棋、日本将棋等）；
2. 无监督学习：自对弈训练方式使得模型能够不断地自我学习和提升，无需任何人类先验知识；
3. 搜索-学习协同优化：MCTS 为神经网络提供监督信号，神经网络指导 MCTS 高效搜索。

然而，AlphaZero 架构也存在一些挑战和局限性。例如，在处理复杂的游戏时，训练过程需要大量的计算资源和时间。此外，模型的可解释性较差。

2.2.1 卷积神经网络

在基于 AlphaZero 架构的 8×8 黑白棋对弈模型中，卷积神经网络 (Convolutional Neural Network, CNN) 扮演着至关重要的角色。它被设计用来学习黑白棋棋盘上的复杂模式和策略，以预测每一步的落子概率和当前局面的胜负价值。我们构建的 ReversiNNet 类定义了这个神经网络的具体结构。

ReversiNNet 类实现卷积神经网络，结构如下：

$$\begin{aligned} \text{输入} &\xrightarrow{\text{Conv2d}(64, 3 \times 3)} \text{ReLU} \xrightarrow{\text{Conv2d}(128, 3 \times 3)} \text{ReLU, Dropout}(0.3) \\ &\xrightarrow{\text{Flatten, FC}(128 \times 8 \times 8, 256)} \text{ReLU, Dropout}(0.3) \xrightarrow{\text{FC}(256, 8 \times 8)} \text{policy} \\ &\xrightarrow{\text{FC}(256, 1), \tanh} \text{value} \end{aligned}$$

输入层 输入层接收当前黑白棋棋盘的状态作为输入。

在 8×8 的黑白棋中，棋盘状态可以用一个二维数组表示，每个位置有三种可能的状态：黑子、白子或空位。在代码实现中，输入的形状为 (batch, 1, 8, 8)，其中 batch 表示批量大小；1 表示通道数，这里我们将棋盘状态看作一个单通道的图像； 8×8 则对应棋盘的大小。

卷积层 卷积层是 CNN 的核心组成部分，它能够自动提取输入数据中的局部特征。在 ReversiNNet 中，我们使用了两个卷积层。

```
1 # 第一个卷积层
2 self.conv1 = nn.Conv2d(1, 64, kernel_size=3, padding=1)
```

该卷积层输出 64 个特征图，卷积核的大小为 3×3 ，padding=1 表示在输入数据的边界周围填充 1 个像素，以保持输出特征图的尺寸与输入相同。

```
1 # 第二个卷积层
2 self.conv2 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
```

第二个卷积层接收第一个卷积层的输出作为输入，因此输入通道数为 64。输出通道数设置为 128，进一步提取更高级的特征。同样，卷积核大小为 3×3 ，填充为 1。

卷积操作的数学公式可以表示为：

$$y_{i,j}^k = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x_{i+m,j+n}^l \cdot w_{m,n}^{l,k} + b^k$$

其中， $y_{i,j}^k$ 是第 k 个输出特征图中位置 (i,j) 的值； $x_{i+m,j+n}^l$ 是第 l 个输入特征图中位置 $(i+m,j+n)$ 的值； $w_{m,n}^{l,k}$ 是卷积核中位置 (m,n) 对应从第 l 个输入通道到第 k 个输出通道的权重； b^k 是第 k 个输出通道的偏置； M 和 N 分别是卷积核的高度和宽度。

在卷积层之后，我们使用了 ReLU（Rectified Linear Unit）激活函数来引入非线性。ReLU 函数的定义为： $\text{ReLU}(x) = \max(0, x)$ 它的作用是将负数输入置为 0，保留正数输入，从而增加网络的表达能力。

全连接层 全连接层将卷积层提取的特征进行整合和变换，以输出最终的预测结果。在 ReversiNNet 中，我们使用了三个全连接层。

```

1 # 将卷积层的输出展平为一维向量
2 x = x.view(-1, self.fc1.in_features)
3 # 第一层
4 self.fc1 = nn.Linear(128 * 8 * 8, 256)
5 x = F.relu(self.fc1(x))
6 # 第二层
7 self.fc2 = nn.Linear(256, 64)
8 policy_logits = self.fc2(x)
9 # 第三层
10 self.fc3 = nn.Linear(256, 1)
11 value = torch.tanh(self.fc3(x))

```

首先，将卷积层的输出展平为一维向量；然后，经过第一个全连接层：输入特征数为 $128 \times 8 \times 8$ ，这是因为第二个卷积层输出的特征图大小为 $128 \times 8 \times 8$ 的矩阵，展平后得到的向量长度为 $128 \times 8 \times 8$ 。输出特征数设置为 256，即该全连接层将输入向量映射到一个 256 维的向量空间。

接着，经过第二个全连接层：该全连接层将 256 维的向量映射到 64 维的向量空间，对应 8×8 棋盘上的 64 个位置，用于输出每个位置的落子概率的对数。最后，经过第三个全连接层：该全连接层将 256 维的向量映射到一个标量，用于输出当前局面的胜负价值。使用 tanh 函数将输出值限制在 $[-1, 1]$ 范围内，其中 -1 表示当前局面必输，1 表示当前局面必胜。全连接层

的数学公式可以表示为：

$$y_j = \sum_{i=0}^{n-1} x_i \cdot w_{i,j} + b_j$$

其中， y_j 是输出向量中第 j 个元素的值； x_i 是输入向量中第 i 个元素的值； $w_{i,j}$ 是连接输入第 i 个元素和输出第 j 个元素的权重； b_j 是第 j 个输出元素的偏置； n 是输入向量的长度。

输出层 输出层包含两个分支：策略头和价值头。策略头输出每个位置的落子概率分布。softmax 函数的定义为：

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=0}^{n-1} \exp(x_j)}$$

其中， x_i 是输入向量中的第 i 个元素， n 是输入向量的长度。softmax 函数将输入向量的每个元素转换为一个概率值，且所有概率值之和为 1。通过对 policy_logits 应用 softmax 函数，将其转换为概率分布。价值头输出当前局面的胜负价值，已经通过 tanh 函数将其限制在 $[-1, 1]$ 范围内。

2.2.2 蒙特卡洛树搜索

蒙特卡洛树搜索（Monte Carlo Tree Search, MCTS）是一种基于模拟的搜索算法，用于在搜索空间中找到最优的落子策略。在黑白棋对弈中，搜索空间非常大，因为每一步都有多个可能的落子选择，而且游戏的深度也很大。MCTS 通过在搜索树中进行随机模拟，逐步构建一个搜索树，并根据模拟结果来评估每个节点的价值，从而选择最优的落子。

节点选择 从根节点开始，根据 UCB（Upper Confidence Bound）公式选择具有最大 UCB 值的子节点，直到到达叶子节点。UCB 公式的作用是平衡探索和利用，既考虑节点的平均价值，又鼓励探索未被充分访问的节点。

UCB 公式定义如下：

$$u(s, a) = Q(s, a) + c_{\text{PUCT}} \cdot P(s, a) \cdot \sqrt{\frac{\sum_{a'} N(s, a')}{1 + N(s, a)}}$$

其中， s 表示当前状态， a 表示动作（即落子位置）； $Q(s, a)$ 为动作 a 在状态 s 下的平均价值， $P(s, a)$ 为卷积神经网络预测的先验概率， $N(s, a)$ 为动作 a 在状态 s 下的节点访问次数， c_{PUCT} 为探索系数。

节点扩展 如果叶子节点不是终止节点，则扩展该节点，生成所有可能的子节点，并计算每个子节点的先验概率。先验概率由神经网络的输出得到。

当搜索到达叶子节点时，通过神经网络预测得到策略 p 和价值 v ，并生成子节点。对于合法落子 a ，其概率被重新归一化：

$$p(a) = \frac{\exp(p_{\text{net}}(a))}{\sum_{a'} \exp(p_{\text{net}}(a'))}$$

模拟 从扩展后的叶子节点开始，进行模拟游戏，直到游戏结束。模拟过程中，根据神经网络的输出选择落子，表达式为：

$$z = \text{rollout}(s, \pi_{\text{sim}})$$

其中， z 表示模拟结果（胜负值）， π_{sim} 表示模拟策略（通常为贪心策略）。

反向传播 将模拟游戏的结果反向传播到根节点，更新每个节点的访问次数和价值。可用如下公式表示：

$$\begin{aligned} N(s, a) &\leftarrow N(s, a) + 1 \\ Q(s, a) &\leftarrow Q(s, a) + \frac{1}{N(s, a)}(z - Q(s, a)) \end{aligned}$$

最终落子 在进行了一定次数的 MCTS 模拟后，根据每个子节点的访问次数来选择最终的落子。通常选择访问次数最多的子节点对应的落子位置。

2.2.3 自对弈训练

自对弈是 AlphaZero 算法的核心训练方式之一。在自对弈过程中，模型自己与自己进行对弈，通过不断地尝试不同的落子策略，生成大量的训练数据。

数据生成 通过自我博弈生成训练数据，每局博弈记录状态 s_t 、策略 π_t 和终局结果 z 。具体流程如下：

1. 初始化棋局状态 s_0
2. 对每个时间步 t :
 - (a) 使用 MCTS 生成策略 π_t
 - (b) 按概率选择落子 $a_t \sim \pi_t$

(c) 执行动作得到新状态 s_{t+1}

3. 终局时计算胜负结果 $z \in \{-1, 0, 1\}$

强化学习更新 使用交叉熵损失和均方误差损失联合优化网络参数 θ :

$$\mathcal{L}(\theta) = \underbrace{(z - v)^2}_{\text{价值损失}} + \underbrace{\left(- \sum_a \pi(a) \log p(a) \right)}_{\text{策略损失}} + \underbrace{c \|\theta\|^2}_{\text{正则化}}$$

其中, z 是终局结果 (-1/0/1), v 是神经网络预测的价值估值, π 是 MCTS 生成的目标概率分布, p 是神经网络预测的策略分布, c 为 L2 正则化系数。

2.3 模块化设计

2.3.1 模型的建立

数据生成 数据生成机制通过自对弈循环生成训练数据,核心函数为 `self_play(game, nnet, num_games)`, 其工作流程如下:

1. 初始化棋局: 调用 `ReversiGame()` 创建初始棋盘;
2. 循环对弈:
 - (a) 使用 `MCTS.get_move_probabilities()` 获取落子概率;
 - (b) 按概率选择落子 `np.random.choice()`;
 - (c) 记录状态 `state`、策略 `pi`、当前玩家 `current_player`;
3. 终局处理: 根据 `get_winner()` 计算胜负结果 z , 生成训练样本 (`state`, `pi`, z)

2.3.2 GUI 架构

人机交互 `ReversiGUI` 类实现人机交互界面:

1. 棋盘渲染:
 - (a) `draw_board()`: 绘制 8×8 网格;
 - (b) `update_board()`: 刷新棋子显示;
 - (c) 记录状态 `state`、策略 `pi`、当前玩家 `current_player`;
2. 交互逻辑:
 - (a) `human_move(event)`: 处理鼠标点击;
 - (b) `ai_move()`: 调用 MCTS 生成 AI 落子;

3. 状态同步：
 - (a) `check_game_over()`: 检测终局状态；
 - (b) `check_and_switch()`: 切换玩家逻辑。

2.3.3 模型的训练和评估

模型训练 模型训练模块由 `train` 函数实现，其核心逻辑如下：

1. 通过 `self_play` 函数生成自对弈数据，包含棋盘状态、落子概率和胜负结果
2. 构造如下损失函数：

$$\mathcal{L}(\theta) = (z - v)^2 - \sum_a \pi(a) \log p(a) + c \|\theta\|^2$$

3. 使用 Adam 优化器，学习率采用指数衰减策略：

$$lc = 0.001 \cdot 0.1^{\text{iter}/\text{num_iter}}$$

4. 自对弈迭代训练。

人机对弈

1. 初始化：
 - (a) 用户选择先手 (1/-1)；
 - (b) 加载预训练模型 `reversi_model.pth`；
2. 交替落子：
 - (a) 人类操作：
 - i. 点击棋盘生成坐标 (i,j)；
 - ii. 合法性验证 `get_valid_moves()`；
 - iii. 执行落子 `execute_move()`；
 - (b) AI 操作：
 - i. 调用 `MCTS.get_move_probabilities()` 生成策略；
 - ii. 选择最优落子 `np.argmax()`；
 - iii. 执行落子并更新棋盘；
3. 终局处理：
 - (a) 计算棋子数 `np.sum(board == 1/-1)`；
 - (b) 显示结果对话框。

2.4 程序框图

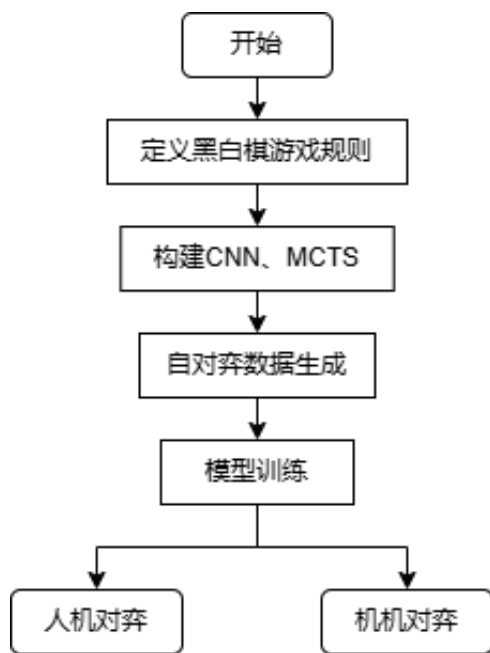


图 1: 程序框图

具体代码实现在附录中。

3 实验结果与所遇问题

3.1 实验结果分析

本实验基于 AlphaZero 架构，在 8×8 棋盘上实现了一个黑白棋对弈模型，成功在零数据的情况下，于 NVIDIA GeForce RTX 3050 Ti 设备上完成训练，并通过自对弈训练提升棋力，具有一定的智能，甚至能达成先手击败黑白棋的人类初学者的成就。

经过在网上查找资料以及我自己在网上在线平台的对弈，我认为黑白棋先手方优势较大。此外，我还在知乎上看到了“**金角银边**”的说法，即，率先占据棋盘四边或四角的选手也会获得较大优势（但是刚刚上手的初学者不懂这些）。下面的图片（图2a2b2c）是人机对弈的结果展示，值得一提的

是，一位志愿者同学后手输给了使用参数 `iter25-epoch16` 训练的 AI 模型（特别地，此模型是我训练所得模型中最强的一个）。

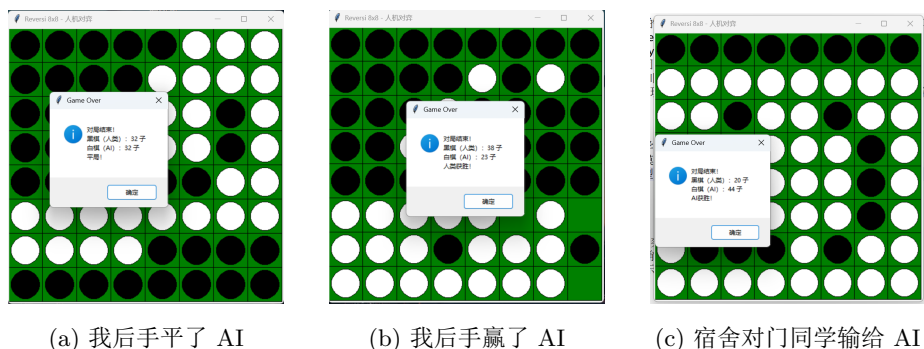


图 2: 人机对弈战况示例

下面是我对于最优参数选择的探索。在本次实验中，我控制总迭代次数为 400 次，训练出四种不同参数组合的模型并让他们分别对弈（各自先手 10 局），并选用“胜率”与“平均胜子数”作为衡量两个模型棋力强弱为标准。从下图（图3a3b3c）的结果可以看出，`iter25-epoch16` 的模型是最强的。因此，我们可以得出这样的结论：在较小迭代次数的前提下，训练模型时自对弈轮数比每轮的迭代次数更重要。

胜	平	负	胜率(%)	平均每局胜子
玩家1(reversi_model-iter10-epoch40-1r):	0	0	20	0.00
玩家2(reversi_model-iter16-epoch25-1r):	20	0	0	100.00

(a) iter10-epoch40 vs iter16-epoch25

胜	平	负	胜率(%)	平均每局胜子
玩家1(reversi_model-iter16-epoch25-1r):	0	0	10	0.00
玩家2(reversi_model-iter20-epoch20-1r):	10	0	0	100.00

(b) iter16-epoch25 vs iter20-epoch20

胜	平	负	胜率(%)	平均每局胜子
玩家1(reversi_model-iter20-epoch20-1r):	10	0	10	50.00
玩家2(reversi_model-iter25-epoch16-1r):	10	0	10	50.00

(c) iter20-epoch20 vs iter25-epoch16

胜	平	负	胜率(%)	平均每局胜子
玩家1(reversi_model-iter16-epoch125):	25	0	25	50.00
玩家2(reversi_model-iter20-epoch20):	25	0	25	50.00

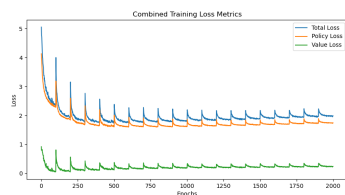
(d) 过拟合的证据

3.2 问题与解决措施

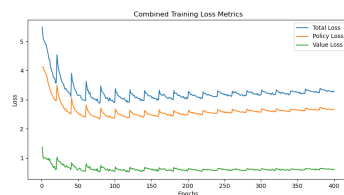
事实上，在实验初期，我使用的迭代次数为 2000 次，分别尝试了 25iter-80epoch、20iter-100epoch、16iter-125epoch、10iter-200epoch 这四种组合，但四者的学习曲线都明显地表现出过拟合现象（如图4a），而且经过机机对弈，也可证明其训练效果远不如迭代次数 400 的模型（如图3d）。

为了解决模型的过拟合问题，我调整了每轮的自对弈局数（5->10），增加了 MCTS 深度（50->200），使用衰减学习率（指数衰减）等方式，较为

有效的解决了此问题。相应地，模型的训练时间也大大增加，训练一次模型大概需要 1h-1.5h。



(a) 自对弈 20 轮，每轮迭代 100 次，MCTS 深度为 100，学习率为 0.001



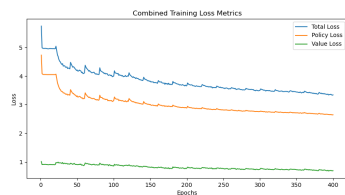
(b) 自对弈 20 轮，每轮迭代 20 次，MCTS 深度为 100，学习率为 0.001



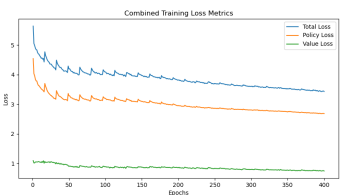
(c) 自对弈 10 轮，每轮迭代 40 次，MCTS 深度为 200，使用衰减学习率



(d) 自对弈 16 轮，每轮迭代 25 次，MCTS 深度为 200，使用衰减学习率



(e) 自对弈 20 轮，每轮迭代 20 次，MCTS 深度为 200，使用衰减学习率



(f) 自对弈 25 轮，每轮迭代 16 次，MCTS 深度为 200，使用衰减学习率

参考文献

- [1] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmasharan Kumar, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm, 2017.

A 附录：实验代码

A.1 Reversi_model.py

此程序用于定义黑白棋游戏逻辑、构建神经网络模型、实现 MCTS 算法、自对弈数据生成和模型训练。

基本设置：

```
1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 import torch.nn.functional as F
6 from tqdm import tqdm
7 import matplotlib.pyplot as plt
8
9 # 设置设备
10 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
11 print("Using device:", device)
12
13 # 将棋盘大小改为 8x8
14 BOARD_SIZE = 8
```

定义黑白棋游戏规则：

```
1 class ReversiGame:
2     def __init__(self):
3         self.board = self.init_board()
4         self.current_player = 1 # 黑棋先行
5
6     def init_board(self):
7         # 初始化棋盘状态
8         board = np.zeros((BOARD_SIZE, BOARD_SIZE), dtype=int)
9         mid1 = BOARD_SIZE // 2 - 1
10        mid2 = BOARD_SIZE // 2
11        # 初始局面：中心四格，黑白对置
12        board[mid1, mid1] = -1
13        board[mid1, mid2] = 1
14        board[mid2, mid1] = 1
```

```

15     board[mid2, mid2] = -1
16     return board
17
18 def get_valid_moves(self, board, player):
19     # 获取当前玩家在给定棋盘状态下的所有合法落子位置
20     moves = []
21     for i in range(BOARD_SIZE):
22         for j in range(BOARD_SIZE):
23             if board[i, j] == 0 and self.check_move(board, player, (i,
24                 j)):
25                 moves.append((i, j))
26     return moves
27
28 def check_move(self, board, player, move):
29     # 检查某个落子是否合法
30     i, j = move
31     valid = False
32     for di in [-1, 0, 1]:
33         for dj in [-1, 0, 1]:
34             if di == 0 and dj == 0:
35                 continue
36             x, y = i + di, j + dj
37             found_opponent = False
38             while 0 <= x < BOARD_SIZE and 0 <= y < BOARD_SIZE:
39                 if board[x, y] == -player:
40                     found_opponent = True
41                 elif board[x, y] == player:
42                     if found_opponent:
43                         valid = True
44                     break
45                 else:
46                     break
47             x += di
48             y += dj
49     return valid
50
51 def execute_move(self, board, player, move):
52     # 在棋盘上执行某个落子操作
53     new_board = board.copy()

```

```

53     i, j = move
54     new_board[i, j] = player
55     for di in [-1, 0, 1]:
56         for dj in [-1, 0, 1]:
57             if di == 0 and dj == 0:
58                 continue
59             pieces_to_flip = []
60             x, y = i + di, j + dj
61             while 0 <= x < BOARD_SIZE and 0 <= y < BOARD_SIZE:
62                 if new_board[x, y] == -player:
63                     pieces_to_flip.append((x, y))
64                 elif new_board[x, y] == player:
65                     for pos in pieces_to_flip:
66                         new_board[pos] = player
67                     break
68                 else:
69                     break
70             x += di
71             y += dj
72     return new_board
73
74 def is_terminal(self, board):
75     # 当双方都无合法走子时结束
76     if len(self.get_valid_moves(board, 1)) == 0 and
77         len(self.get_valid_moves(board, -1)) == 0:
78         return True
79     return False
80
81 def get_winner(self, board):
82     # 获取胜者
83     black = np.sum(board == 1)
84     white = np.sum(board == -1)
85     if black > white:
86         return 1
87     elif white > black:
88         return -1
89     else:
90         return 0

```

```

91 def display_board(self, board):
92     # 使用 matplotlib 显示棋盘状态 (1: 黑, -1: 白, 0: 空)
93     plt.figure(figsize=(4, 4))
94     plt.imshow(board, cmap='gray_r')
95     plt.xticks(range(BOARD_SIZE))
96     plt.yticks(range(BOARD_SIZE))
97     plt.show()

```

构建神经网络模型:

```

1 class ReversiNet(nn.Module):
2     def __init__(self, board_size, dropout_p=0.3):
3         super(ReversiNet, self).__init__()
4         self.board_size = board_size
5         self.conv1 = nn.Conv2d(1, 64, kernel_size=3, padding=1)
6         self.conv2 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
7         self.fc1 = nn.Linear(128 * board_size * board_size, 256)
8         self.fc_policy = nn.Linear(256, board_size * board_size)
9         self.fc_value = nn.Linear(256, 1)
10        self.dropout = nn.Dropout(p=dropout_p)
11
12    def forward(self, x):
13        # 输入 x: (batch, 1, board_size, board_size)
14        x = F.relu(self.conv1(x))
15        x = self.dropout(x)
16        x = F.relu(self.conv2(x))
17        x = self.dropout(x)
18        x = x.view(-1, 128 * self.board_size * self.board_size)
19        x = F.relu(self.fc1(x))
20        x = self.dropout(x)
21        policy = self.fc_policy(x) # 原始 logits
22        value = torch.tanh(self.fc_value(x))
23        return policy, value

```

构建 MCTS 辅助决策:

```

1 class MCTSNode:
2     def __init__(self, board, player, parent=None):
3         self.board = board

```

```

4         self.player = player
5         self.parent = parent
6         self.children = {} # move -> child node
7         self.visit_count = 0
8         self.total_value = 0
9         self.prior = 0
10
11
12     def softmax(x):
13         e_x = np.exp(x - np.max(x))
14         return e_x / e_x.sum()
15
16
17     class MCTS:
18         def __init__(self, game, nnet, c_puct=1.0, num_simulations=200):
19             self.game = game
20             self.nnet = nnet
21             self.c_puct = c_puct
22             self.num_simulations = num_simulations
23             self.Q = {} # key: (node_id, move) -> Q值
24             self.N = {} # key: (node_id, move) -> 访问次数
25             self.P = {} # key: (node_id, move) -> 先验概率
26
27         def search(self, node):
28             # 执行蒙特卡洛树搜索
29             if self.game.is_terminal(node.board):
30                 winner = self.game.get_winner(node.board)
31                 if winner == 0:
32                     return 0
33                 return 1 if winner == node.player else -1
34
35             valid_moves = self.game.get_valid_moves(node.board, node.player)
36             if not valid_moves:
37                 # 无合法走子则“过子”，转换玩家
38                 next_board = node.board.copy()
39                 child = MCTSNode(next_board, -node.player, parent=node)
40                 value = -self.search(child)
41                 return value
42

```

```

43     if not node.children:
44         # 扩展叶子节点
45         board_tensor =
46             torch.FloatTensor(node.board).unsqueeze(0).unsqueeze(0).to(device)
47         policy_logits, value = self.nnet(board_tensor)
48         policy_logits = policy_logits.detach().cpu().numpy().flatten()
49         policy = np.zeros(BOARD_SIZE * BOARD_SIZE)
50         for move in valid_moves:
51             idx = move[0] * BOARD_SIZE + move[1]
52             policy[idx] = np.exp(policy_logits[idx])
53         if policy.sum() > 0:
54             policy = policy / np.sum(policy)
55         else:
56             policy = np.ones(BOARD_SIZE * BOARD_SIZE) / (BOARD_SIZE *
57                 BOARD_SIZE)
58         for move in valid_moves:
59             next_board = self.game.execute_move(node.board,
60                 node.player, move)
61             child_node = MCTSNode(next_board, -node.player,
62                 parent=node)
63             node.children[move] = child_node
64             key = (id(node), move)
65             self.P[key] = policy[move[0] * BOARD_SIZE + move[1]]
66             self.N[key] = 0
67             self.Q[key] = 0
68         return value.item()
69
70     # 选择具有最大 UCB 值的走法
71     best_score = -float('inf')
72     best_move = None
73     for move in valid_moves:
74         key = (id(node), move)
75         if self.N[key] > 0:
76             u = self.Q[key] + self.c_puct * self.P[key] *
77                 np.sqrt(node.visit_count) / (1 + self.N[key])
78         else:
79             u = self.c_puct * self.P[key] * np.sqrt(node.visit_count +
80                 1e-8)
81         if u > best_score:

```

```

76         best_score = u
77         best_move = move
78
79     child = node.children[best_move]
80     value = -self.search(child)
81     key = (id(node), best_move)
82     self.N[key] += 1
83     self.Q[key] = (self.Q[key] * (self.N[key] - 1) + value) /
        self.N[key]
84     node.visit_count += 1
85     return value
86
87 def get_move_probabilities(self, root, temp=1):
88     # 获取根节点下每个合法落子的概率分布
89     for _ in range(self.num_simulations):
90         self.search(root)
91     valid_moves = self.game.get_valid_moves(root.board, root.player)
92     counts = np.array([self.N.get((id(root), move), 0) for move in
        valid_moves])
93     if len(valid_moves) == 0:
94         return valid_moves, np.array([])
95     if temp == 0:
96         best_move = valid_moves[np.argmax(counts)]
97         probs = np.zeros(len(valid_moves))
98         probs[valid_moves.index(best_move)] = 1
99         return valid_moves, probs
100     counts = counts ** (1. / temp)
101     probs = counts / np.sum(counts)
102     return valid_moves, probs

```

自对弈数据生成和模型训练:

```

1 def self_play(game, nnet, num_games=10):
2     examples = []
3     for _ in range(num_games):
4         game_instance = ReversiGame()
5         states, mcts_probs, current_players = [], [], []
6         while True:
7             # 若当前玩家无合法走子, 则直接过子

```

```

8         valid = game.get_valid_moves(game_instance.board,
9                                         game_instance.current_player)
10
11     if not valid:
12         states.append(game_instance.board.copy())
13         mcts_probs.append(np.zeros(BOARD_SIZE * BOARD_SIZE))
14         current_players.append(game_instance.current_player)
15         game_instance.current_player =
16             -game_instance.current_player
17         # 如果双方均无合法走子, 则结束游戏
18         if len(game.get_valid_moves(game_instance.board,
19                                     game_instance.current_player)) == 0:
20             winner = game.get_winner(game_instance.board)
21             rewards = [1 if winner == player else -1 if winner !=
22                        0 else 0 for player in current_players]
23             for state, pi, reward in zip(states, mcts_probs,
24                                         rewards):
25                 examples.append((state, pi, reward))
26             break
27         continue
28
29     root = MCTS(game, nnet, num_simulations=200)
30     node = MCTSNode(game_instance.board,
31                     game_instance.current_player)
32     moves, probs = root.get_move_probabilities(node, temp=1)
33     # 若 MCTS 返回空列表, 则同样进行过子操作
34     if len(moves) == 0:
35         states.append(game_instance.board.copy())
36         mcts_probs.append(np.zeros(BOARD_SIZE * BOARD_SIZE))
37         current_players.append(game_instance.current_player)
38         game_instance.current_player =
39             -game_instance.current_player
40         continue
41
42     # 构造落子概率向量 (长度 = BOARD_SIZE * BOARD_SIZE)
43     pi = np.zeros(BOARD_SIZE * BOARD_SIZE)
44     for move, p in zip(moves, probs):
45         idx = move[0] * BOARD_SIZE + move[1]
46         pi[idx] = p
47     states.append(game_instance.board.copy())

```

```

40     mcts_probs.append(pi)
41     current_players.append(game_instance.current_player)
42     # 按概率选择走法
43     move = moves[np.random.choice(len(moves), p=probs)]
44     game_instance.board = game.execute_move(game_instance.board,
45                                             game_instance.current_player, move)
46     game_instance.current_player = -game_instance.current_player
47     if game.is_terminal(game_instance.board):
48         winner = game.get_winner(game_instance.board)
49         rewards = [1 if winner == player else -1 if winner != 0
50                  else 0 for player in current_players]
51         for state, pi, reward in zip(states, mcts_probs, rewards):
52             examples.append((state, pi, reward))
53         break
54     return examples
55
56 def train(nnet, examples, lr, epochs=10, batch_size=32,
57          return_losses=False):
58     # 采用参数衰减
59     optimizer = optim.Adam(nnet.parameters(), lr=lr, weight_decay=1e-4)
60     epoch_losses = []
61     epoch_policy_losses = []
62     epoch_value_losses = []
63
64     # 显示每个 epoch 的训练过程
65     for epoch in tqdm(range(epochs), desc="Training epochs"):
66         np.random.shuffle(examples)
67         batch_losses = []
68         batch_policy_losses = []
69         batch_value_losses = []
70         for i in range(0, len(examples), batch_size):
71             batch = examples[i:i + batch_size]
72             boards = np.array([ex[0] for ex in batch])
73             target_pi = np.array([ex[1] for ex in batch])
74             target_v = np.array([ex[2] for ex in batch])
75             boards = torch.FloatTensor(boards).unsqueeze(1).to(device) #
76                 (batch, 1, BOARD_SIZE, BOARD_SIZE)
77             target_pi = torch.FloatTensor(target_pi).to(device)

```

```

75         target_v = torch.FloatTensor(target_v).unsqueeze(1).to(device)
76         optimizer.zero_grad()
77         out_pi, out_v = nnet(boards)
78         loss_v = F.mse_loss(out_v, target_v)
79         loss_pi = -torch.mean(torch.sum(target_pi *
            F.log_softmax(out_pi, dim=1), dim=1))
80         loss = loss_v + loss_pi
81         loss.backward()
82         optimizer.step()
83         batch_losses.append(loss.item())
84         batch_policy_losses.append(loss_pi.item())
85         batch_value_losses.append(loss_v.item())
86         avg_loss = np.mean(batch_losses)
87         avg_policy_loss = np.mean(batch_policy_losses)
88         avg_value_loss = np.mean(batch_value_losses)
89         epoch_losses.append(avg_loss)
90         epoch_policy_losses.append(avg_policy_loss)
91         epoch_value_losses.append(avg_value_loss)
92         print(f"Epoch {epoch+1}/{epochs}: Loss={avg_loss:.4f}, Policy
            Loss={avg_policy_loss:.4f}, Value Loss={avg_value_loss:.4f}")
93
94     if return_losses:
95         return epoch_losses, epoch_policy_losses, epoch_value_losses

```

主函数:

```

1  if __name__ == "__main__":
2      game = ReversiGame()
3      nnet = ReversiNet(BOARD_SIZE, dropout_p=0.3).to(device)
4      all_examples = []
5      num_iterations = 20 # 自对弈轮数
6      global_epoch_losses = []
7      global_epoch_policy_losses = []
8      global_epoch_value_losses = []
9
10     for iter in range(num_iterations):
11         # 采用指数衰减
12         lr = lr0 * (0.1 ** (iter / num_iterations))
13

```

```

14     print(f"Self play iteration: {iter + 1} with learning rate:
15           {lr}")
16     examples = self_play(game, nnet, num_games=10) # 自对弈盘数
17     all_examples.extend(examples)
18     # 每次自对弈后进行训练，并返回该轮训练的各 epoch loss
19     epoch_losses, epoch_policy_losses, epoch_value_losses =
20         train(nnet, all_examples, lr=lr, epochs=20, batch_size=32,
21             return_losses=True)
22
23     global_epoch_losses.extend(epoch_losses)
24     global_epoch_policy_losses.extend(epoch_policy_losses)
25     global_epoch_value_losses.extend(epoch_value_losses)
26
27     # 保存训练好的模型
28     torch.save(nnet.state_dict(), "reversi_model-iter20-epoch20-lr.pth")
29
30     # 将所有轮次的训练 loss 画在一张图上
31     total_epochs = len(global_epoch_losses)
32     epochs_range = range(1, total_epochs + 1)
33     plt.figure(figsize=(10, 5))
34     plt.plot(epochs_range, global_epoch_losses, label="Total Loss")
35     plt.plot(epochs_range, global_epoch_policy_losses, label="Policy
36             Loss")
37     plt.plot(epochs_range, global_epoch_value_losses, label="Value Loss")
38     plt.xlabel("Epochs")
39     plt.ylabel("Loss")
40     plt.title("Combined Training Loss Metrics")
41     plt.legend()
42     plt.savefig("combined_training_metrics-iter20-epoch20-lr.png")
43     plt.show()

```

A.2 play.py

本程序构建了 GUI 界面，可以调用训练好的模型实现人机对弈。
基础配置：

```

1 import tkinter as tk
2 from tkinter import messagebox

```

```

3 import numpy as np
4 import torch
5 from Reversi_model_8_gpu import ReversiGame, ReversiNNet, BOARD_SIZE
6
7 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
8 print("Using device:", device)

```

设置 GUI 界面:

```

1 class ReversiGUI:
2     def __init__(self, master, first_player=1):
3         self.master = master
4         self.master.title("Reversi 8x8 - 人机对弈")
5         self.game = ReversiGame() # 8x8 棋盘
6         self.nnet = ReversiNNet(BOARD_SIZE).to(device)
7         # 加载模型
8         self.nnet.load_state_dict(torch.load("reversi_model-iter20-epoch20-lr.pth",
9             map_location=device))
10        self.nnet.eval()
11        self.cell_size = 60 # 每个格子的像素大小
12        self.canvas = tk.Canvas(self.master, width=BOARD_SIZE *
13            self.cell_size, height=BOARD_SIZE * self.cell_size)
14        self.canvas.pack()
15        self.canvas.bind("<Button-1>", self.human_move)
16        self.game_over = False # 对局是否结束
17        self.game.current_player = first_player # 设置初始执棋方
18        self.draw_board()
19        self.update_board()
20        # 开局前检测当前方是否有合法走子，无则切换
21        self.check_and_switch()
22
23        # 若 AI 先手，主动触发 AI 落子
24        if first_player == -1:
25            self.master.after(500, self.ai_move)
26
27    def draw_board(self):
28        # 绘制棋盘网格
29        for i in range(BOARD_SIZE):
30            for j in range(BOARD_SIZE):

```

```

29         x0 = j * self.cell_size
30         y0 = i * self.cell_size
31         x1 = x0 + self.cell_size
32         y1 = y0 + self.cell_size
33         self.canvas.create_rectangle(x0, y0, x1, y1, fill="green",
                                     outline="black")
34
35     def update_board(self):
36         # 更新棋盘上棋子的显示，并检查对局是否结束
37         self.canvas.delete("piece")
38         board = self.game.board
39         for i in range(BOARD_SIZE):
40             for j in range(BOARD_SIZE):
41                 x0 = j * self.cell_size + 5
42                 y0 = i * self.cell_size + 5
43                 x1 = (j + 1) * self.cell_size - 5
44                 y1 = (i + 1) * self.cell_size - 5
45                 if board[i, j] == 1:
46                     self.canvas.create_oval(x0, y0, x1, y1, fill="black",
                                              tags="piece")
47                 elif board[i, j] == -1:
48                     self.canvas.create_oval(x0, y0, x1, y1, fill="white",
                                              tags="piece")
49         self.master.update()
50         self.check_game_over()
51
52     def check_game_over(self):
53         # 若双方均无合法走子则对局结束，弹窗显示黑白棋子数和结果
54         if self.game_over:
55             return
56         if self.game.is_terminal(self.game.board):
57             self.game_over = True
58             board = self.game.board
59             black_count = np.sum(board == 1)
60             white_count = np.sum(board == -1)
61             if black_count > white_count:
62                 result = "人类获胜！"
63             elif white_count > black_count:
64                 result = "AI获胜！"

```

```

65         else:
66             result = "平局！"
67             msg = f"对局结束！\n黑棋（人类）：{black_count}
68                 子\n白棋（AI）：{white_count} 子\n{result}"
69             messagebox.showinfo("Game Over", msg)
70
71     def check_and_switch(self):
72         # 检查当前执棋方是否有合法走子；若无则自动切换到对方
73         valid_moves = self.game.get_valid_moves(self.game.board,
74             self.game.current_player)
75         if not valid_moves:
76             self.game.current_player = -self.game.current_player
77             self.update_board()
78             if self.game.current_player == -1:
79                 self.master.after(500, self.ai_move)
80
81     def human_move(self, event):
82         # 仅当当前走子方为人类（黑棋）且对局未结束时响应点击
83         if self.game.current_player != 1 or self.game_over:
84             return
85         valid_moves = self.game.get_valid_moves(self.game.board, 1)
86         if not valid_moves:
87             self.game.current_player = -1
88             self.update_board()
89             self.master.after(500, self.ai_move)
90             return
91         j = event.x // self.cell_size
92         i = event.y // self.cell_size
93         move = (i, j)
94         if move in valid_moves:
95             self.game.board = self.game.execute_move(self.game.board, 1,
96                 move)
97             self.game.current_player = -1
98             self.update_board()
99             self.master.after(500, self.ai_move)
100
101     def ai_move(self):
102         # 仅当当前走子方为 AI（白棋）且对局未结束时进行落子
103         if self.game.current_player != -1 or self.game_over:

```

```

101         return
102     valid_moves = self.game.get_valid_moves(self.game.board, -1)
103     if not valid_moves:
104         self.game.current_player = 1
105         self.update_board()
106         return
107     from Reversi_model_8_gpu import MCTS, MCTSNode #
108         确保此模块中的代码也使用GPU（已在其内部处理）
109     root = MCTS(self.game, self.nnet, num_simulations=400)
110     node = MCTSNode(self.game.board, -1)
111     moves, probs = root.get_move_probabilities(node, temp=0)
112     if not moves:
113         self.game.current_player = 1
114         self.update_board()
115         return
116     move = moves[np.argmax(probs)]
117     self.game.board = self.game.execute_move(self.game.board, -1,
118         move)
119     self.game.current_player = 1
120     self.update_board()
121     self.check_and_switch()

```

主函数:

```

1 if __name__ == "__main__":
2     root = tk.Tk()
3     # 1 表示人类先手, -1 表示 AI 先手
4     first_player = int(input("请输入先手方 (1 表示人类先手, -1 表示 AI
5         先手): "))
6     gui = ReversiGUI(root, first_player)
7     root.mainloop()

```

A.3 AI_vs_AI.py

本程序用于进行两个模型之间的机机对弈，测试训练出的两个模型的棋力大小。

AI Players 的配置:

```

1 import numpy as np
2 import torch
3 from Reversi_model_8_gpu import ReversiGame, ReversiNet, BOARD_SIZE,
    MCTS, MCTSNode, device
4
5 MODEL1_NAME = "reversi_model-iter25-epoch80-reg" #
    玩家1模型名称 (字符串变量)
6 MODEL2_NAME = "reversi_model-iter20-epoch100-reg" #
    玩家2模型名称 (字符串变量)
7 MODEL1_PATH = MODEL1_NAME + ".pth" # 玩家1使用的模型文件路径
8 MODEL2_PATH = MODEL2_NAME + ".pth" # 玩家2使用的模型文件路径

```

设置轮换规则和终止规则，计算战绩：

```

1 def simulate_game(black_net, white_net, num_simulations=400):
2     """
3     模拟一局对弈：传入执黑和执白的神经网络，采用 MCTS 搜索进行落子，
4     当一方无合法走子时直接切换回合，直至游戏结束，返回最终棋盘状态。
5     """
6     game = ReversiGame()
7     while not game.is_terminal(game.board):
8         valid_moves = game.get_valid_moves(game.board,
9             game.current_player)
10        if not valid_moves:
11            # 当前玩家无合法走子，直接过子
12            game.current_player = -game.current_player
13            continue
14        # 根据当前执棋方选择对应的网络
15        net = black_net if game.current_player == 1 else white_net
16
17        # 采用 MCTS 搜索选择落子 (温度 temp=0 表示贪心选择)
18        root = MCTS(game, net, num_simulations=num_simulations)
19        node = MCTSNode(game.board, game.current_player)
20        moves, probs = root.get_move_probabilities(node, temp=0)
21        if len(moves) == 0:
22            game.current_player = -game.current_player
23            continue
24        chosen_move = moves[np.argmax(probs)]

```

```

25     game.board = game.execute_move(game.board, game.current_player,
26                                     chosen_move)
27
28     return game.board
29
30 def evaluate_game(board):
31     """
32     根据最终棋盘状态计算双方棋子数、胜负结果以及胜子差：
33     返回：
34     result: (result_black, result_white)
35     其中 result_black 为1表示黑棋胜、-1表示黑棋负、0表示平局；
36     result_white 则为相反。
37     margin_black: 黑棋比白棋多的棋子数；
38     margin_white: 白棋比黑棋多的棋子数；
39     black_count, white_count: 棋子数
40     """
41     black_count = np.sum(board == 1)
42     white_count = np.sum(board == -1)
43     if black_count > white_count:
44         result = (1, -1)
45     elif black_count < white_count:
46         result = (-1, 1)
47     else:
48         result = (0, 0)
49     margin_black = black_count - white_count
50     margin_white = white_count - black_count
51     return result, margin_black, margin_white, black_count, white_count
52
53 def main():
54     num_games_each_config = 10 # 每种配置下对弈局数（共两种配置）
55
56     # 统计两模型的对局结果
57     stats = {
58         "model1": {"wins": 0, "draws": 0, "losses": 0, "margin": 0.0,
59                  "games": 0},
60         "model2": {"wins": 0, "draws": 0, "losses": 0, "margin": 0.0,
61                  "games": 0}
62     }

```

```

61
62 # 加载模型
63 model1_obj = ReversiNet(BOARD_SIZE).to(device)
64 model2_obj = ReversiNet(BOARD_SIZE).to(device)
65 model1_obj.load_state_dict(torch.load(MODEL1_PATH,
66                                     map_location=device))
67 model2_obj.load_state_dict(torch.load(MODEL2_PATH,
68                                     map_location=device))
69
70 model1_obj.eval()
71 model2_obj.eval()
72
73 # 配置1: model1_obj执黑（先手），model2_obj执白（后手）
74 for _ in range(num_games_each_config):
75     board = simulate_game(model1_obj, model2_obj,
76                           num_simulations=400)
77     (result_black, result_white), margin_black, margin_white, _, _ =
78         evaluate_game(board)
79     # 更新统计: 黑棋由model1_obj执, 白棋由model2_obj执
80     stats["model1"]["games"] += 1
81     stats["model2"]["games"] += 1
82     if result_black == 1:
83         stats["model1"]["wins"] += 1
84         stats["model2"]["losses"] += 1
85     elif result_black == -1:
86         stats["model1"]["losses"] += 1
87         stats["model2"]["wins"] += 1
88     else:
89         stats["model1"]["draws"] += 1
90         stats["model2"]["draws"] += 1
91     stats["model1"]["margin"] += margin_black
92     stats["model2"]["margin"] += margin_white
93
94 # 配置2: model2_obj执黑（先手），model1_obj执白（后手）
95 for _ in range(num_games_each_config):
96     board = simulate_game(model2_obj, model1_obj,
97                           num_simulations=400)
98     (result_black, result_white), margin_black, margin_white, _, _ =
99         evaluate_game(board)
100     # 此时黑棋由model2_obj执, 白棋由model1_obj执

```

```

94     stats["model2"]["games"] += 1
95     stats["model1"]["games"] += 1
96     if result_black == 1:
97         stats["model2"]["wins"] += 1
98         stats["model1"]["losses"] += 1
99     elif result_black == -1:
100         stats["model2"]["losses"] += 1
101         stats["model1"]["wins"] += 1
102     else:
103         stats["model2"]["draws"] += 1
104         stats["model1"]["draws"] += 1
105     stats["model2"]["margin"] += margin_black
106     stats["model1"]["margin"] += margin_white
107
108     # 计算胜率和平均胜子量
109     for key in stats:
110         games = stats[key]["games"]
111         win_rate = (stats[key]["wins"] / games) * 100 if games > 0 else 0
112         avg_margin = stats[key]["margin"] / games if games > 0 else 0
113         stats[key]["win_rate"] = win_rate
114         stats[key]["avg_margin"] = avg_margin
115
116     # 打印对局结果
117     header = "      胜    平    负    胜率(%)    平均每局胜子"
118     line1 = f"玩家1({MODEL1_NAME}): {stats['model1']['wins']:3d}
119             {stats['model1']['draws']:3d} {stats['model1']['losses']:3d}
120             {stats['model1']['win_rate']:7.2f}
121             {stats['model1']['avg_margin']:7.2f}"
122     line2 = f"玩家2({MODEL2_NAME}): {stats['model2']['wins']:3d}
123             {stats['model2']['draws']:3d} {stats['model2']['losses']:3d}
124             {stats['model2']['win_rate']:7.2f}
125             {stats['model2']['avg_margin']:7.2f}"
126
127     print(header)
128     print(line1)
129     print(line2)

```

主函数，调用程序：

```

1 if __name__ == "__main__":

```

2

`main()`
